

Optimized Lightweight Hardware Trojan-based Fault Attack on DES

Fan Zhang, Yiran Zhang

College of Information Science and Electronic Engineering,
Institute of Cyber Security Research,
Zhejiang University,
Hangzhou, P.R.China,
fanzhang@zju.edu.cn

Shengwen Shi, Shize Guo

Department of Information Security,
Institute of North Electronic Equipment,
Beijing, P.R.China,
shishengwenem@163.com

Ziyuan Liang, Samiya Qureshi

College of Information Science and Electronic Engineering,
Institute of Cyber Security Research,
Zhejiang University,
Hangzhou, P.R.China,
liangziyuan@zju.edu.cn

Congyuan Xu

College of Information Science
and Electronic Engineering,
Zhejiang University,
Hangzhou, P.R.China,
cyxu@zju.edu.cn

Abstract—An optimized lightweight Hardware Trojan (HT) based fault attack is proposed, especially for those resource-constrained environments such as IoT networks. Firstly, *Algebraic fault analysis* (AFA) is introduced to evaluate different fault models and search for the optimal one. Next, considering the limited resource, a lightweight HT is carefully designed which only flips one bit of the circuit in IoT device. Finally, AFA is applied again to exploit the fault and recover the secret key. An illustrative attack is demonstrated on DES implemented on an FPGA platform, SASEBO-GII. This paper shows that, for single bit fault injection at different rounds or different indexes in the same round, the reduced key search space of DES varies. The proposed technique can search for the optimal fault model, guide the lightweight Hardware Trojan design and automatically recover the secret key. Only one fault is required to recover the secret key of DES, which improves the stealthiness of the designed Hardware Trojan in IoT networks. The entire attack framework can also be applied to other block ciphers such as AES and PRESENT.

Index Terms—Fault Injection, Lightweight Hardware Trojan, Algebraic Fault Analysis, DES

I. INTRODUCTION

Along with the development of Internet of Things (IoT), the security of encryption chips used in IoT networks has now been acquiring attentions from the public. Hardware Trojan (HT) based fault attacks have emerged as a challenging security concern about IoT encryption chips because it can corrupt and introduce malicious modifications of electronic hardware. An adversary can mount such an attack to cause undesired functional behavior or to leak secret information

This work was supported in part by the National Natural Science Foundation of China under Grant 61472357 and 61571063, in part by the Open Fund of State Key Laboratory of Cryptology, and in part by the Zhejiang University Fundamental Research Funds for the Central Universities under Grant 2018QNA5005.

from inside a cryptographic chip [1], [2], e.g., the master key. Critically, in IoT networks, the adversary might use the leaked information from Hardware Trojans to deduce the manufacture key. Different from Software Trojan, Hardware Trojan is usually inserted into an IC before it is fabricated and is virtually impossible to remove after fabrication. In addition, due to their stealthy nature, activating arbitrary HTs and observing their effects can be extremely challenging.

Hardware Trojan can raise undesired functional behavior of an IC, or provide covert channels where sensitive information could leak. This paper mainly focuses on the latter one, *i.e.*, emphasis on covert channels with security measurements. A Hardware Trojan is usually composed of a trigger logic for activation and a payload logic for IC modification. The covert channel for HT should be as rebellious and hard to detect. Current research studies show that thermal radiation, power consumption, electromagnetic radiation and a faulty output can be utilized for new covert channel construction and payload logic design.

Recent research on Hardware Trojan mainly motivates on how to model and detect HT [3]–[7]. In [4], HT is proposed to leak a secret key from inside a cryptographic IC through power side-channels. The *differential power analysis* (DPA) is applied to analyze about 10000 power traces and extract the master key. Even though the IC has been designed and proposed to minimize side-channel leakage, a hardware modification could defeat the protection and extract the secret information. Additionally, our proposed work includes improving HT detection [6], [7], amplifying HT leakages [3], exploiting new covert channels [5], etc. To reduce the data complexity, some researchers proposed to combine HT with fault attacks. Firstly, Hardware Trojan is designed to inject an ideal fault and corrupt the computation of IC. Secondly, the

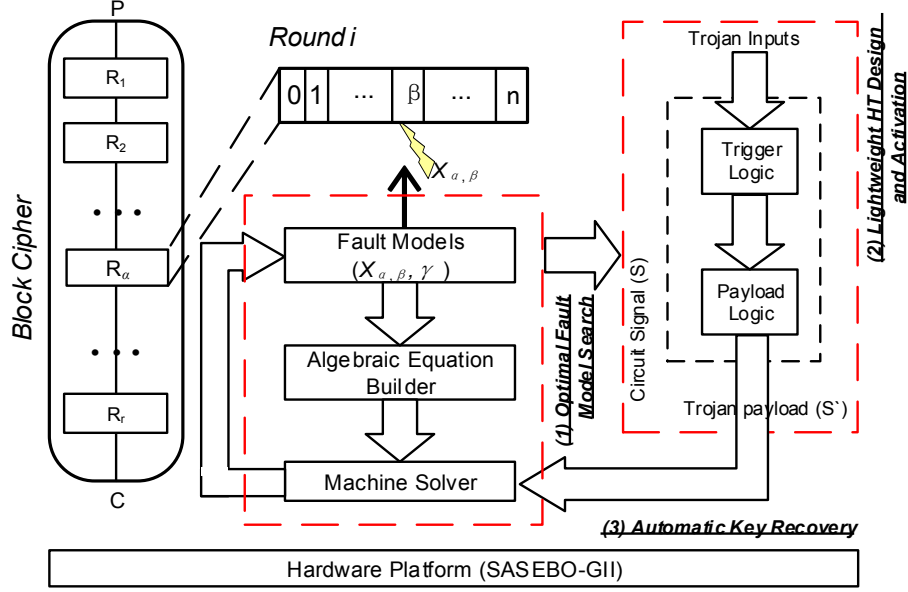


Fig. 1. An overview of the proposed framework.

classic differential fault analysis (DFA) is applied to analyze the faulty outputs and recover the secret key [8], [9]. Compared with power leakage-based HT, fault-based HT is very efficient and effective. Only one or very few of fault injections are required. Compared with traditional fault injection techniques such as lasers and clock-glitching [10], a well designed HT can inject any type of faults at a low cost. Moreover, it can precisely control the timing and location of the fault injection.

In [8], [9], Hardware Trojan is designed to flip a nibble or byte of intermediate states. The fault width is dependent on the well studied fault models used in DFAs on block ciphers. A robust fault model for Hardware Trojan can guarantee the its stealthiness. It is important that the fault width should be as small as possible (e.g., single bit is desired), so that the HT is lightweight. In addition to that, the data complexity of the attack should be minimized, thus the off-line key analysis could be simplified. All of these previous fault-based HT attacks rely on a forehand knowledge of the fault model. To find an automatic approach to search for optimal fault models and guide the HT design is very important to enhance the efficiency and stealthy of HT, which is main motivation of our proposed work.

This article proposed an optimized Hardware Trojan based fault attack on block ciphers with algebraic techniques. Firstly, *Algebraic fault analysis* (AFA) is introduced to enumerate different fault models and evaluate the residual key entropy of block cipher. Furthermore, a lightweight Hardware Trojan is carefully designed which only flips one bit of the circuit to inject the ideal fault. Finally, AFA is applied again to exploit the fault and recover the secret key. A practical attack is demonstrated for DES on the standard evaluation board SASEBO-GII. Experimental results show that AFA can be

used to search for the best injection location and find a balanced tradeoff between the cost and precision for fault. The result can guide the design of Hardware Trojan and make it much more efficient. As to DES as a target, only one fault injection is required to recover the secret key.

The rest of the article is organized as follows. Section II proposes the optimized HT-based fault attack framework. Section III describes the cipher of DES and represents AFA on DES. Section IV provides the experiment results of our attack on DES. Section V briefly talks about the discussion on the cipher PRESENT. Section VI concludes the paper.

II. THE ATTACK FRAMEWORK ON GENERAL BLOCK CIPHERS

Let P, C, n, r denote the plaintext, ciphertext, block size, and the number of rounds for a block cipher $\mathbb{B}$, respectively. $X_{\alpha, \beta}$ stands for an intermediate state of n bits. γ denotes the fault width. With the assumption that there is only one state for each round, α is the index of the round and β is the index of n/γ possible fault locations in the state. The assumption also holds in many hardware implementations of block ciphers. $X_{\alpha, \beta}$ is the position of the fault injection, thus the fault model can be described as $\mathbb{F} = \{X_{\alpha, \beta}, \gamma\}$. The proposed optimized fault-based Hardware Trojan attack framework is shown in Fig.1.

A. Optimal Fault Models Search

Fault model has significant effect on the efficiency of fault attacks. It is desired that Hardware Trojan should be stealthy and lightweight, in order to avoid the detections and to reduce the implementation cost. An optimal fault model $\mathbb{F}$ for Hardware Trojan is designed such that it should minimize both the fault width and the data complexity. How to find

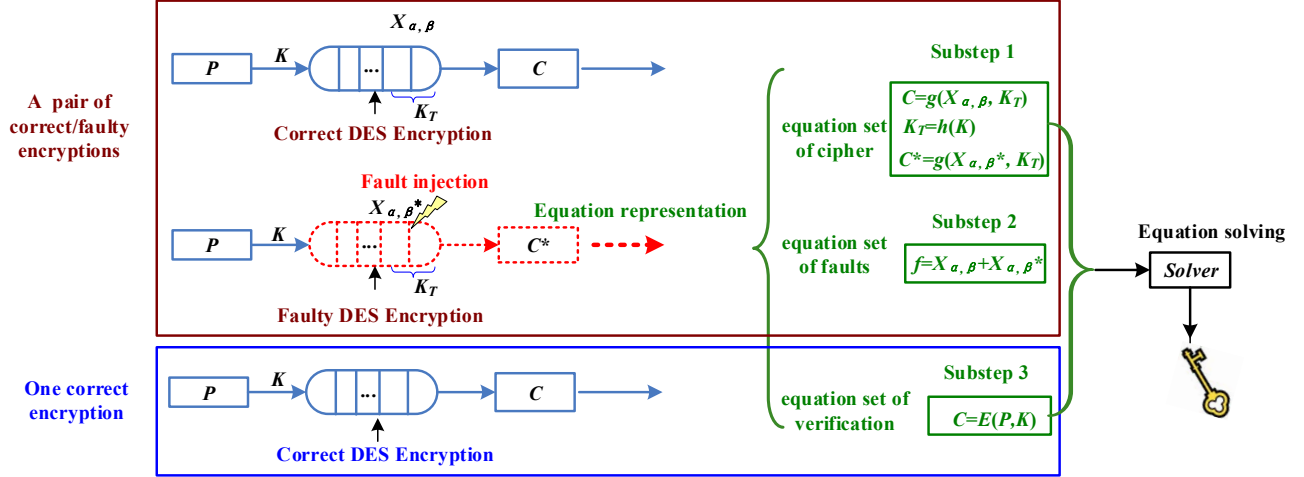


Fig. 2. Key recovery procedure.

the optimal fault model with different parameters of α, β, γ is critical to the design of HT. Traditional DFAs require manual analysis on the fault propagation path for each model. They are practically challenging if there are massive fault models to compute or if the search for the injection point goes to deep rounds. Fault models derived from DFAs may not be optimal because they strongly depend on the subjective capabilities of the adversary. In contrast, we introduce *algebraic fault analysis* (AFA) to enumerate each possible fault model and search for the optimal case. For each enumeration, a full AFA is conducted that consists of three steps:

- Inducing a fault: For a given fault model $\mathbb{F} = \{X_{\alpha, \beta}, \gamma\}$, a fault of γ bits is induced to $X_{\alpha, \beta}$ in block cipher $\mathbb{B}$ in a simulator.
- Constructing the equations for $\mathbb{B}$ and $\mathbb{F}$: An automatic tool is leveraged to generate algebraic equations for both the cipher and the fault. The details can be referred to as in [11].
- Solving the equations: A machine solver can be applied to solve the system of equations and retrieve the secret key.

Note that examining for the optimal model for Hardware Trojan can be estimated with three metrics: the remained key entropy $\phi(K)$ (represent how many bits can be recovered based on the theory of information), the solving time T and the success rate ρ . Whether to use only one of three metrics or to use all of them as the evaluation standard is highly dependent on the practical requirement.

B. Lightweight HT Design and Activation

With the optimal fault model, a lightweight Hardware Trojan can be designed to inject fault into a circuit. A typical HT is composed of a trigger logic and a payload logic denoted as TL and PL respectively [1]. TL is to active the payload logic and PL is to inject faults. The designed principle requires TL to be a simple component with a low activation rate. In this

attack framework, two possible suggestions are implemented to satisfy the design needs, especially on the trigger logic. One is based on the customized combinational logic comparing the input and output pins. The other is based on the embedded sensors (if available) to trigger the Hardware Trojan.

Commonly, the whole cryptographic IC is well implemented and not easy to modify. However, the output register is an exception as it has to output useful information through IO pins. In our proposed HT design, we use the output register which stores the ciphertext, as it will only be updated at the end of the full encryption. The trigger logic is a comparison of two q -bit values: one is in the plaintext, while the other is in the output register. PL can be activated only when they are equal. Note that the q -bit value does not need to be continuous, which means only the connection relation of corresponding wires needs to be changed. It will not increase the complexity of the design. Taking $q = 4$ as a simple example, one possible trigger condition could be represented as :

$$\{Out[3], Out[25], Out[18], Out[7]\} \\ == \{In[9], In[15], In[2], In[27]\} \quad (1)$$

where Out denotes the value of output register (here is the ciphertext), In denotes the input value. Compared to the sequential circuits design [12], [13], this type of proposed TL is just a simple combinational logic and no logic explicitly exports the key to the output register. The feedback structure also makes the trigger condition dynamic. All these facts provide certain stealthiness. Note that the activation rate will be highly dependent on the size of q , which is about 2^{-q} .

Another supplementary choice is to utilize the thermal sensor embedded on the circuit to conduct the triggering, which is available on many of FPGA chips. To trigger the hardware trajon, a hairdryer can be used to heat the entire chip. When the chip temperature reaches the specific threshold, a trigger signal will be generated from the thermal sensor and served for further usage.

To minimize the circuit size, the design of payload logics fulfills single bit-based fault model. Such PL is only responsible to flip one bit of the original circuit. It is implemented as an XOR gate with LUTs in FPGA. The precise location of the injection is the ideal bit line to be flipped. The activation of TL will determine whether the designed HT is bypassed and whether a single-bit fault is induced.

C. Automatic Offline Key Recovery

With the faults injected, the desired faulty ciphertext can be collected. The algebraic equations of $\mathbb{B}$ and optimal $\mathbb{F}$, can be built and fed to the machine solver for key recovery. Many solvers, such as mutantXL algorithm, Gröbner basis-based, SAT-based solver can be leveraged. Fig.2 illustrates the overall key recovery procedure.

In Fig.2, $X_{\alpha,\beta}$ and $X_{\alpha,\beta}^*$ are the intermediate states of correct and faulty encryption respectively. With or without the fault injected by the HT, we can get the faulty ciphertext C^* and correct ciphertext C . Then, we can convert the fault model, the cipher, and the pair of faulty and correct encryptions into algebraic equation sets. Finally, a machine solver is used for key recovery.

III. ALGEBRAIC REPRESENTATIONS FOR DES AND THE FAULT

A. Description of DES

DES is a block cipher that was selected by the US National Bureau of Standards in 1976 for data encryption. It uses a 56-bit key (represented on 64 bits including 8 parity check bits) and operates on 64-bit blocks. DES has an iterative structure which applies the same round transformation F . The 16 rounds are preceded by a bit-permutation IP and followed by the inverse bit-permutation IP^{-1} (FP). Each round is parameterized by a 48-bit round key K_r derived from the master key. A ciphertext C is computed from a plaintext P as follows:

$$C = IP^{-1} \circ (\bigcirc_{r=1}^{16} F_{K_r}) \circ IP(P) \quad (2)$$

The round transformation follows a Feistel scheme, namely, the 64-bit block is split into two 32-bit parts L (the left part) and R (the right part), and F is defined as:

$$F_{K_r} = (R, L \oplus f_{K_r}(R)) \quad (3)$$

where f is a function parameterized with a 48-bit key and operating on a 32-bit block. The output block of the r -th round shall be denoted as (L_r, R_r) . Defining $(L_0, R_0) = IP(P)$, we have $(L_r, R_r) = F_{K_r}(L_{r-1}, R_{r-1})$ for every $r \leq 16$ and $C = IP^{-1}(L_{16}, R_{16})$.

The function f first applies an expansion layer E that expands the 32 input bits into 48 output bits by duplicating 16 of them. The round key is then introduced by bitwise addition afterward the block is split into eight 6-bit blocks, each entering into a different S-Box S_i producing a 4-bit output. Finally, the 32 bits from the eight S-Box outputs are permuted through a bit-permutation P which yields the 32-bit output block.

B. Why targeting at DES

The proposed framework of optimized fault-based Hardware Trojan attack can be applied to different block ciphers. Among them, our recommended work uses DES as the illustration for several reasons. DES is conventional and could be easily understood. Its hardware implementation code is available from many open sources. The most important reason is that there is slight difference among the output bits of DES S-Box (6-bit in and 4-bit out), which allows us to demonstrate how our proposed framework can find a better position for the Trojan within the same round. Note that, in terms of the fault-based Hardware Trojan, the optimal location is also highly associated with the specific round. Choosing DES also reflects the flexibility of the proposed framework when it enumerates all possible indexes inside the same round and balances the trade-off between different indexes in different rounds. The extended applications to other block ciphers will be further treated in future work. The lightweight block cipher PRESENT might be a better target as it is based on bitwise permutations.

C. Building the Equation Set of AFA on DES

Let α denote the position of the round where faults are injected into the left part of the DES internal state at the end of the round. The equation set of AFA on DES is shown in Algorithm 1.

Algorithm 1: The equation set of AFA on DES

```

1 (1) Building DES key schedule equations
2  $(K_1, K_2, \dots, K_{16}) \leftarrow \text{KeySchedule}(K)$ 
3 (2) Building correct DES encryption equations
4  $C \leftarrow IP(C)$ 
5  $\text{Swap } C = L_{16} || R_{16}$ 
6 for  $r = 16; r > 0; r--$  do
7    $R_{r-1} \leftarrow L_r$ 
8    $L_{r-1} \leftarrow f(K_r, R_{r-1}) + R_r$ 
9 end
10  $P \leftarrow IP^{-1}(L_0 || R_0)$ 
11 (3) Building faulty DES encryption equations
12  $C^* \leftarrow IP(C^*)$ 
13  $\text{Swap } C^* = L_{16}^* || R_{16}^*$ 
14 for  $r = 16; r > \alpha; r--$  do
15    $R_{r-1}^* \leftarrow L_r^*$ 
16    $L_{r-1}^* \leftarrow f(K_r, R_{r-1}^*) + R_r^*$ 
17 end
18 (4) Building the fault equations
19  $\text{Nonzero} \leftarrow L_\alpha + L_\alpha^*$ 

```

In Algorithm 1, the most difficult part is to build the equations for the S-Box in the f function and to build the equation for the faults.

1) *Build the equations for the S-Box:* We use the techniques in [14] to represent every S-Box output bit with high-degree equations from the six S-Box input bits. The illustration of how to represent the first bit of the first S-Box in DES is shown in Eq. (4).

$$\begin{aligned}
y_1 = & 1 + x_1 + x_2 + x_3 + x_4 + x_6 + x_3x_4 + x_4x_5 \\
& + x_1x_5 + x_1x_6 + x_1x_3x_5 + x_4x_5x_6 \\
& + x_2x_4x_5 + x_1x_2x_5 + x_1x_3x_4 + x_3x_4x_5 \\
& + x_1x_4x_5 + x_2x_5x_6 + x_1x_4x_6 + x_1x_3x_4x_5 \quad (4) \\
& + x_1x_2x_4x_5 + x_1x_2x_3x_5 + x_1x_3x_5x_6 \\
& + x_1x_4x_5x_6 + x_1x_2x_3x_6 + x_1x_2x_3x_4x_5 \\
& + x_1x_2x_3x_4x_6.
\end{aligned}$$

In Eq. (4), $x_1, x_2, \dots, x_6$ denoted as the input of the first S-Box and y_1 denoted as the first bit of output.

2) *Build the equations for the fault*: Let X denote the λ -bit correct data unit of DES. $X = x_1 \| x_2 \| \dots \| x_\lambda$. In this paper, X represent a 32-bit left state register in encryption ($\lambda = 32$). Y denotes the faulty value of X . $Y = y_1 \| y_2 \| \dots \| y_\lambda$. There are m possible locations for the injected faults where $m = \lambda/\gamma$. Z denotes the fault difference of X and Y :

$$Z = z_1 \| z_2 \| \dots \| z_\lambda, z_i = x_i + y_i, 1 \leq i \leq \lambda \quad (5)$$

Z can be divided into m parts: $Z_1 \| Z_2 \| \dots \| Z_m$,

$$\begin{aligned}
Z_i = & z_{\gamma \times (i-1) + 1} \| z_{\gamma \times (i-1) + 2} \| \dots \| z_{\gamma \times i}, \\
& 1 \leq i \leq m \quad (6)
\end{aligned}$$

Since t is known, Z can be denoted as

$$Z_i = 0, 1 \leq i \leq m, i \neq t \quad (7)$$

Z_t has a non-zero value of w -bits. We introduce a single bit variable u_t to represent that Z_t is faulty.

$$\begin{aligned}
u_t = & (1 \oplus z_{w \times (t-1) + 1}) (1 \oplus z_{w \times (t-1) + 2}) \dots \\
& (1 \oplus z_{\gamma \times t}) = 0 \quad (8)
\end{aligned}$$

Using Eq. (7) and (8), Z can be represented with $\gamma + 1$ variables and $\gamma(m + 1) + 2$ CNF equations.

IV. PHYSICAL EXPERIMENTS ON DES

A. Experiment Setup

Hardware setup. We take DES implementation on a SASEBO-GII FPGA board [15] as the target and verify the feasibility of the proposed framework. SASEBO-GII has several operation modes. In our experiments, we use the mode that both the control FPGA and cryptographic FPGA are occupied. The board is connected to a PC with a USB interface. In one encryption, the PC generate a plaintext and send it to the control FPGA via USB cable. The control FPGA is configured to convert the USB data to bitstream that cryptographic FPGA can handle directly. The cryptographic FPGA is the carrier of DES. After cryptographic FPGA finishes the encryption, the ciphertext is sent back to the PC.

Software setup. In the first stage of our framework, we construct a simulator for DES encryption and fault injection. An automatic tool is customized for AFA on DES under different fault models. To minimize the HT design, γ is set as 1. A machine solver, CryptoMiniSAT v2.9.4 [16], runs on a PC

with Intel Core i5-4460 @3.2GHz and 4GBytes RAM. Two modes of using CryptminiSAT are considered in this paper.

Mode A, is with a verification stage. Equations for the full round of correct DES encryption, for the last $16 - \alpha$ rounds of faulty DES encryption and that for the faults $X_\gamma \oplus X_\gamma^*$ are represented and sent to the solver. The number of solutions should be only one all the time. The purpose is to get the statistics of different solving time T with different number of fault injections. A time threshold T_{max} can be set to compute the success rate of the attack ρ . When the solver runs over T_{max} but cannot satisfy all the equations, it can be considered as a failure.

Mode B, is without the verification stage. Equations for the last $16 - \alpha$ rounds of both correct, faulty DES encryption, and that for the faults $X_\gamma \oplus X_\gamma^*$ are represented and sent to the solver. The solver is running at a multiple solution mode. The purpose is to get the statistics of the remained key entropy $\phi(K)$ for different fault models. $\phi(K) = 0$ denotes that K is fully recovered.

B. Optimal Fault Models of Attacking DES

This section shows how to search for parameters of the optimal fault model $\mathbb{F} = \{X_{\alpha, \beta}, \gamma\}$. As did in previous work [17], we assume that fault is injected to the left part of the DES state at the end of one round.

Firstly, to guarantee the stealthiness of HT, the fault width γ should be as small as possible ($\gamma = 1$). Then we need to determine the optimal value of α, β . For this, two modes of using CryptminiSAT are considered for different value of α, β .

1) *AFA on DES under Mode A: Determine α* . In each fault injection, we randomly choose β in the range of $[0, 31]$. When $1 \leq \alpha \leq 10$, or $13 \leq \alpha \leq 16$, 50 times of simulations were conducted for each round, the solver can not output the solution within an acceptable time. Only if $\alpha = 11$ or 12 , we can get the solution. 50 times of simulations was conducted for $\alpha = 11$ or 12 . The results show that $\rho(\alpha = 12) = 71.7\%$ and $T_{avg}(\alpha = 12) = 6.86$ seconds, $\rho(\alpha = 11) = 7.25\%$ and $T_{avg}(\alpha = 11) = 23.4$ seconds ($T_{max} = 30$ s). Taking both success rate and the average solving time into consideration, a single bit injected into the 12-th round might be more efficient.

Determine β . We fix $\alpha = 11$ and $\alpha = 12$, try every possible value of β (in the range of $[0, 31]$). 640 simulations was conducted, where each value of β is tested for 20 times. The statistical results are shown in Fig.3 and Fig.4.

From Fig.3 and Fig.4, we can see that when $\beta \in \{0, 3, 4, 7, 8, 11, 12, 15, 16, 19, 20, 23, 24, 27, 28, 31\}$, the solving time is much smaller than others. We will explain this in the latter part.

2) *AFA on DES under Mode B: Determine α* . In each fault injection, we randomly choose β in the range of $[0, 31]$. When $1 \leq \alpha \leq 10$, the solver cannot output the solution within 24 hours. When $13 \leq \alpha \leq 16$, there are so many solutions and the solver cannot output all of them within 24 hours.

When $\alpha = 12$, 100 simulations were conducted, the solver can output all of the solutions within one hour. The statistical results are shown in Fig.5.

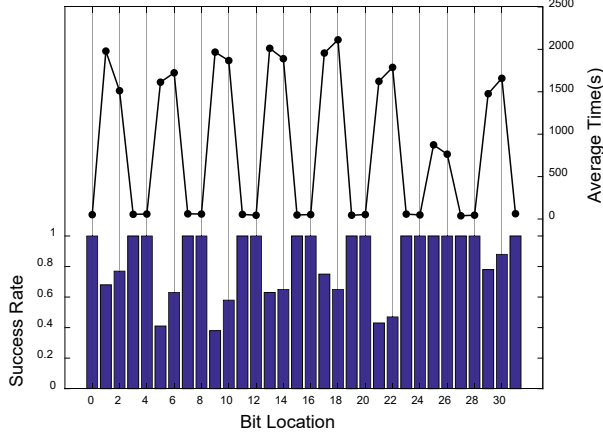


Fig. 3. Solving results in different bits ($\alpha = 11, T_{max} = 3600s$).

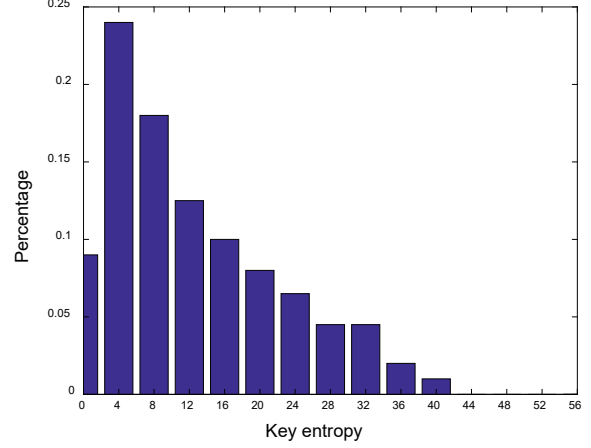


Fig. 5. Statistics on $\phi(K)$, $\alpha = 12$.

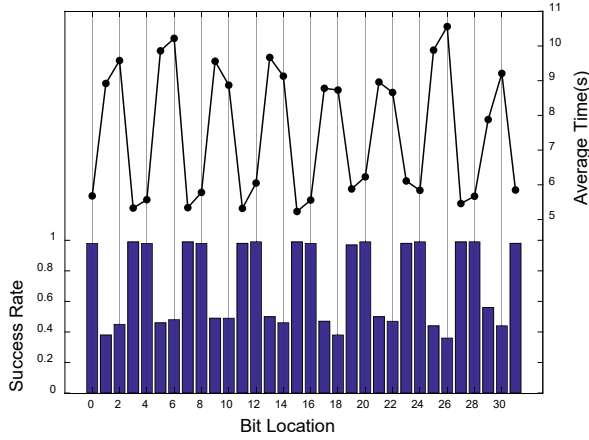


Fig. 4. Solving results in different bits ($\alpha = 12, T_{max} = 30s$).

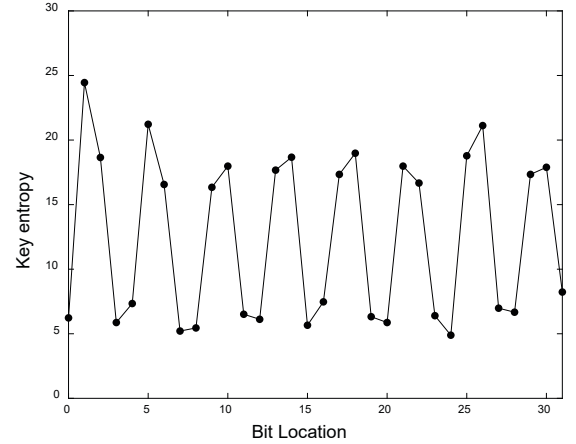


Fig. 6. Statistics on $\phi(K)$, $\alpha = 12$.

Determine β . Then as did in previous sections, we fix $\alpha = 12$ and try every possible value of β ($[0,31]$). 640 simulations were conducted, each value of β is tested for 20 times. The statistical results are shown in Fig.6.

We can see that the result in Fig.6 is consistent to Fig.4. When $\beta \in \{0, 3, 4, 7, 8, 11, 12, 15, 16, 19, 20, 23, 24, 27, 28, 31\}$, $\phi(K)$ is much smaller than others. The reason behind lies on the fact that different from other values of β , according to the expansion layer E , single bit fault of these values can be propagated to two S-Boxes instead of one S-Box.

When $\alpha = 11$, 100 simulations were conducted, the solver can output all of the solutions within one hour. The statistical results are shown in Fig.7.

From Fig.7, we can see that $\phi(K)$ can be reduced to 0 for 82.5% cases, which means that the 56-bit master key of DES can be directly recovered with a high probability. The average value of $\phi(K)$ is 0.40. Then we fix $\alpha = 11$ and try every possible value of β ($[0,31]$). 320 simulations is conducted,

each value of β is tested for 10 times. The statistical results are shown in Fig.8.

We can see the similar results that when $\beta \in \{0, 3, 4, 7, 8, 11, 12, 15, 16, 19, 20, 23, 24, 27, 28, 31\}$, $\phi(K)$ is much smaller than others.

From Mode A and B, we can get the consistent conclusion that $\mathbb{F} = \{X_{11 \text{ or } 12}, \beta, 1\}$ ($\beta \in \{0, 3, 4, 7, 8, 11, 12, 15, 16, 19, 20, 23, 24, 27, 28, 31\}$) is optimal fault position.

C. Hardware Trojan Design and Activation

In the second stage, a lightweight HT is designed for fault injection on DES. The block diagram of the proposed HT is shown in Fig.9. If $V_{active} = 1$, the payload is activated. Since $V_{out}^* = \overline{V_{out}}$, the bit fault is injected. Otherwise, the proposed HT is bypassed.

To decrease the rate of activation and guarantee the stealthiness of the HT, q should be large. According to our experience, q can be appropriately set to 24. The logical test space is

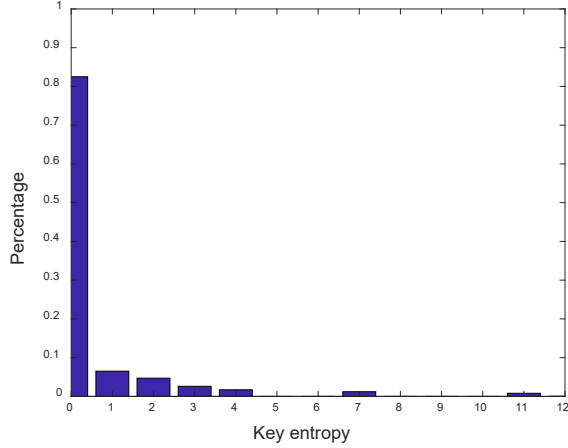


Fig. 7. Statistics on $\phi(K)$, $\alpha = 11$.

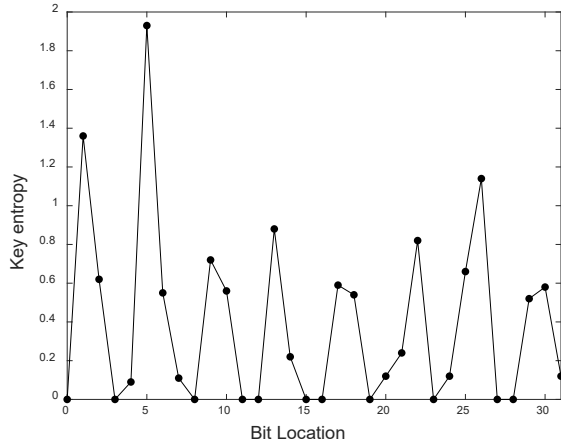


Fig. 8. Statistics on $\phi(K)$, $\alpha = 11$.

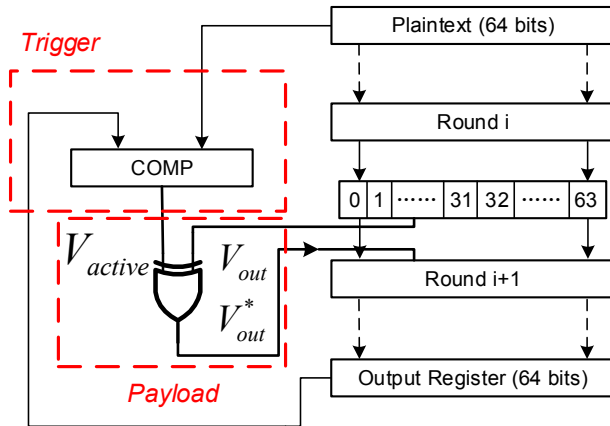


Fig. 9. Block diagram of the proposed HT.

2^{24} and the activation probability is about 2^{-24} if there is no control and the Trojan is accidentally triggered. When $q = 24$, the HT takes 34 LUTs, almost 0.1% of total area of the FPGA where DES is implemented. The injection can be disguised as a random failure as the output register only exposes a ciphertext, not the secret key itself.

In practice, the adversary needs to control when the desired trojan can be activated as he wants. In this paper, we assume the adversary has the capability of sending the specific plaintexts and receiving the corresponding ciphertexts. In order to trigger the Hardware Trojan, he has to conduct three times of encryptions with specific plaintexts.

The logical waveform of the DES core with the HT is shown in Fig.10 where three encryptions are depicted by data valid *Dvld* signal. In the first encryption, a correct ciphertext is observed as `0x3C3A24F4929331A0`. As $q = 24$, activation condition is determined by 24 bits. For the convenience of description, we set trigger condition to $Out[1 : 24] == In[17 : 40]$. To active the HT, the plaintext in the second encryption is intentionally selected as `0x7A853C3A2489239B` by the adversary. Note that this plaintext has the exactly same bit values on the specific positions as the ciphertext from the first encryption. Then the activation condition is satisfied and a faulty ciphertext is received as `0xD5F558EECD55FE6`. In the third encryption, we input the same plaintext again and get the correct ciphertext as `0x606E8E61F64D94A3`. We also verified the feasibility of the HT on the SASEBO-GII FPGA board. The result is the same as above.

D. Key Recovery Analysis

In the last stage, as did in Section 4.2, we convert the faulty ciphertext, the correct ciphertext and DES into algebraic equations. CryptoMiniSAT is used again to recover the secret key of DES.

V. DISCUSSION

As aforementioned in Section.II, the framework for such a fault-based Hardware Trojan attack is actually proposed for the general block ciphers, not just for DES. It can also be applied to other block ciphers when similar Hardware Trojan attacks are required to be mounted on the circuit. For example, PRESENT-80 is a 31-round block cipher with 64-bit block size, 80-bit key size and 4-bit S-box size. In PRESENT-80, $1 \leq \alpha \leq 31, 1 \leq \beta \leq 64$. Using the similar method in Section 4.2.2, our AFA under Mode B can find optimized positions for PRESENT. When $\alpha = 28$, there are four possible values for β , that is, $\beta = 1, 2, 3, 4$, which has the similar remained key entropy $\Phi(K)$. The statistics of $\Phi(K)$ for $\alpha = 28, \beta = 1$ with 100 simulations are listed in Fig.11. $\Phi(K)$ is in the range $[14.2, 34]$ and the average value is 20.1. The average solving time is 30 minutes. The design for such a Hardware Trojan will be left as future work, which should be similar as the one for DES but will be activated with different conditions.

VI. CONCLUSION

This article proposed a framework for optimizing fault-based HT attack on block ciphers with algebraic techniques.

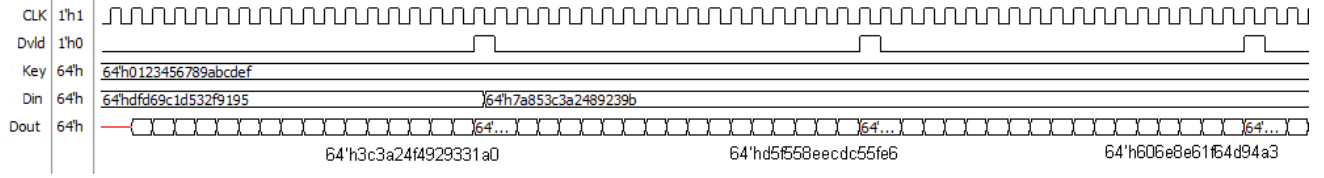


Fig. 10. Logical waveform of the DES core with the HT.

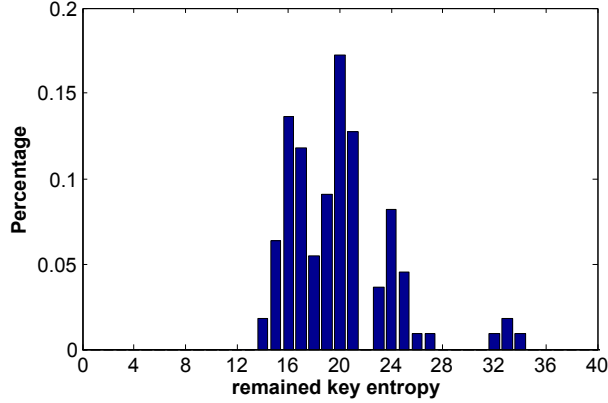


Fig. 11. Statistics on $\phi(K)$ for PRESENT-80, $\alpha = 28, \beta = 1$.

A practical attack is demonstrated for DES on SASEBO-GII. Experimental results show that *Algebraic fault analysis* (AFA) can search for the optimized fault model, guide lightweight HT design and automatically recover the secret key. If single bit is injected into the certain bit at the end of the 12-th round or any bit at the end of the 11-th round of DES, the key entropy can be reduced to 5 and 0, respectively. Only one fault injection is required to recover the secret key of DES. It should be noted that the proposed framework is very general and can be applied to other block ciphers.

REFERENCES

- [1] S. Bhunia, M. S. Hsiao, M. Banga, and S. Narasimhan, "Hardware trojan attacks: Threat analysis and countermeasures," *Proceedings of the IEEE*, vol. 102, no. 8, pp. 1229–1247, 2014.
- [2] M. Tehranipoor and F. Koushanfar, "A survey of hardware trojan taxonomy and detection," 2010.
- [3] J.-F. Gallais, J. Großschädl, N. Hanley, M. Kasper, M. Medwed, F. Regazzoni, J.-M. Schmidt, S. Tillich, and M. Wójcik, "Hardware trojans for inducing or amplifying side-channel leakage of cryptographic software," in *Trusted Systems*. Springer, 2011, pp. 253–270.
- [4] L. Lin, W. Burleson, and C. Paar, "Moles: malicious off-chip leakage enabled by side-channels," in *Proceedings of the 2009 ICCAD*. ACM, 2009, pp. 117–122.
- [5] Y. Liu, Y. Jin, and Y. Makris, "Hardware trojans in wireless cryptographic ICs: silicon demonstration & detection method evaluation," in *Proceedings of the International Conference on Computer-Aided Design*. IEEE Press, 2013, pp. 399–404.
- [6] M. Potkonjak, A. Nahapetian, M. Nelson, and T. Massey, "Hardware trojan horse detection using gate-level characterization," in *Design Automation Conference, 2009. DAC'09. 46th ACM/IEEE*. IEEE, 2009, pp. 688–693.
- [7] H. Salmani, M. Tehranipoor, and J. Plusquellic, "New design strategy for improving hardware trojan detection and reducing trojan activation time," in *Hardware-Oriented Security and Trust, 2009. HOST'09. IEEE International Workshop on*. IEEE, 2009, pp. 66–73.
- [8] A. P. Johnson, S. Saha, R. S. Chakraborty, D. Mukhopadhyay, and S. Gören, "Fault attack on AES via hardware trojan insertion by dynamic partial reconfiguration of FPGA over ethernet," in *Proceedings of the 9th Workshop on Embedded Systems Security*. ACM, 2014, p. 1.
- [9] R. Kumar, P. Jovanovic, W. Burleson, and I. Polian, "Parametric trojans for fault-injection attacks on cryptographic hardware," in *Fault Diagnosis and Tolerance in Cryptography (FDTC), 2014 Workshop on*. IEEE, 2014, pp. 18–28.
- [10] H. Bar-El, H. Choukri, D. Naccache, M. Tunstall, and C. Whelan, "The sorcerer's apprentice guide to fault attacks," *Proceedings of the IEEE*, vol. 94, no. 2, pp. 370–382, 2006.
- [11] F. Zhang, X. Zhao, S. Guo, T. Wang, and Z. Shi, "Improved algebraic fault analysis: A case study on piccolo and applications to other lightweight block ciphers," in *Constructive Side-Channel Analysis and Secure Design*. Springer, 2013, pp. 62–79.
- [12] S. S. Ali, R. S. Chakraborty, D. Mukhopadhyay, and S. Bhunia, "Multi-level attacks: An emerging security concern for cryptographic hardware," in *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2011*. IEEE, 2011, pp. 1–4.
- [13] S. Bhasin, J.-L. Danger, S. Guilley, X. T. Ngo, and L. Sauvage, "Hardware trojan horses in cryptographic IP cores," in *Fault Diagnosis and Tolerance in Cryptography (FDTC), 2013 Workshop on*. IEEE, 2013, pp. 15–29.
- [14] L. R. Knudsen and C. V. Miolane, "Counting equations in algebraic attacks on block ciphers," *International Journal of Information Security*, vol. 9, no. 2, pp. 127–135, 2010.
- [15] T. Katashita, A. Satoh, T. Sugawara, N. Homma, and T. Aoki, "Development of side-channel attack standard evaluation environment," in *Circuit Theory and Design, 2009. ECCTD 2009. European Conference on*. IEEE, 2009, pp. 403–408.
- [16] M. Soos, K. Nohl, and C. Castelluccia, "Extending SAT solvers to cryptographic problems," in *Theory and Applications of Satisfiability Testing-SAT 2009*. Springer, 2009, pp. 244–257.
- [17] M. Rivain, "Differential fault analysis on DES middle rounds," in *Cryptographic Hardware and Embedded Systems-CHES 2009*. Springer, 2009, pp. 457–469.